E4 Educator v2.0

T07

Audio output basics

Tier 1 · Tutorial 7 of 7

Walark Electronics · Fundrum Engineering · May 2026

★ Tier 1 Final Tutorial ★

Completing T07 finishes Tier 1 and unlocks Tier 2 — the TFT display, touch, SD card, WiFi, MQTT, OTA, and ESP-NOW.

In this tutorial

You will learn:

- How the piezo buzzer on the E4 works and how LEDC drives it
- How to play musical notes and simple melodies non-blocking
- How to use the ESP32 DAC for true analog audio output
- How to generate sine waves, sawtooth waves and simple tones via DAC
- How the PWM transistor output on GPIO 12 controls loads
- How to build an audio alert system combining all three outputs
- How to use button events to trigger different sounds and effects
- Tier 1 consolidation — everything from T01 through T07 in one project

You will need:

- E4 Educator v2.0 board with ESP32 fitted
- T01 through T06 completed — Button.h available
- USB-C cable (data-capable)
- Optional: small speaker or earphones connected to DAC_AUDIO (GPIO 25) via 3.5mm header

1 Three audio outputs on the E4

The E4 Educator has three independent audio-capable outputs, each suited to different purposes. Understanding the difference between them is the foundation of this tutorial.

Output	GPIO / Define	Type	Best for
Piezo buzzer	`GPIO 27 / PIEZO`	PWM square wave via LEDC	Alerts, beeps, tones, simple melodies. Loud and attention-grabbing.
DAC audio	`GPIO 25 / DAC_AUDIO`	True 8-bit analog output	Sine waves, smooth tones, waveform generation. Quiet without amplification.
PWM load output	`GPIO 12 / TFT_BL`	PWM via BC847 transistor	Driving loads beyond GPIO current limit. TFT backlight in this course. Motor or relay in future projects.

GPIO 12 is TFT_BL

The PWM transistor output on GPIO 12 is used as the TFT backlight driver in all display projects. In T07 we explore its general PWM capability without the TFT fitted. In Tier 2 projects, always initialise GPIO 12 LOW before LEDC setup as covered in T02.

2 Piezo buzzer — tones and melodies

A piezo buzzer converts electrical oscillation into mechanical vibration and sound. Drive it with a square wave at the right frequency and it produces a tone. The LEDC peripheral on the ESP32 generates this square wave efficiently in hardware — no processor time is wasted toggling the pin in software.

2.1 Note frequencies

Musical notes have specific frequencies in Hz. Here are the most useful ones for embedded alert sounds:

Note	C4	D4	E4	G4	A4
Hz	262	294	330	392	440
Note	**B4**	**C5**	**D5**	**E5**	**G5**
Hz	494	523	587	659	784

2.2 Playing a tone

```
#include <Arduino.h>

const int PIEZO_CH  = 1;   // LEDC channel (0 is used by TFT_BL)
const int PIEZO_RES = 8;   // 8-bit resolution

void toneOn(int frequencyHz) {
  ledcSetup(PIEZO_CH, frequencyHz, PIEZO_RES);
  ledcAttachPin(PIEZO, PIEZO_CH);
  ledcWrite(PIEZO_CH, 128);  // 50% duty cycle = square wave
}

void toneOff() {
```

```
  ledcWrite(PIEZO_CH, 0);    // Duty = 0 = silence
}

void setup() {
  Serial.begin(115200);
  // Play A4 (440Hz) for 500ms then silence
  toneOn(440);
  delay(500);
  toneOff();
  Serial.println("Tone demo complete.");
}

void loop() {}
```

Key point

A square wave at 50% duty cycle (ledcWrite value of 128 for 8-bit) gives the cleanest piezo tone. Lower duty cycles produce quieter but harsher sounds. The LEDC channel for the piezo should be different from channel 0 used for TFT_BL — use channel 1 for PIEZO throughout the course.

2.3 Non-blocking tone player

For alert sounds in real firmware — a beep when a button is pressed, a warning when a threshold is exceeded — you need tones that play without blocking the rest of the firmware. Here is a clean non-blocking tone sequencer:

```
// Tone.h — non-blocking tone sequencer for E4 projects
// Drop into src/ alongside Button.h
#pragma once
#include <Arduino.h>

struct ToneNote {
  int     freqHz;    // 0 = silence / rest
  int     durationMs;
};

class TonePlayer {
public:
  TonePlayer(int pin, int channel = 1, int resolution = 8)
    : _pin(pin), _ch(channel), _res(resolution),
      _playing(false), _noteIdx(0), _noteStart(0),
      _sequence(nullptr), _seqLen(0) {}

  void begin() {
    ledcSetup(_ch, 1000, _res);
    ledcAttachPin(_pin, _ch);
    ledcWrite(_ch, 0);
  }

  // Play a sequence of notes (does not block)
  void play(const ToneNote* seq, int len) {
    _sequence = seq;
    _seqLen   = len;
```

```
    _noteIdx  = 0;
    _playing  = true;
    _noteStart = millis();
    _startNote(0);
  }

  void stop() {
    _playing = false;
    ledcWrite(_ch, 0);
  }

  bool isPlaying() { return _playing; }

  // Call every loop() pass
  void update() {
    if (!_playing) return;
    unsigned long now = millis();
    if (now - _noteStart >= (unsigned long)_sequence[_noteIdx].durationMs) {
      _noteIdx++;
      if (_noteIdx >= _seqLen) {
        stop();
        return;
      }
      _noteStart = now;
      _startNote(_noteIdx);
    }
  }

private:
  int   _pin, _ch, _res;
  bool  _playing;
  int   _noteIdx, _seqLen;
  unsigned long _noteStart;
  const ToneNote* _sequence;

  void _startNote(int idx) {
    int freq = _sequence[idx].freqHz;
    if (freq == 0) {
      ledcWrite(_ch, 0);
    } else {
      ledcSetup(_ch, freq, _res);
      ledcAttachPin(_pin, _ch);
      ledcWrite(_ch, 128);
    }
  }
};
```

Using the TonePlayer:

```
#include "TonePlayer.h"
#include "Button.h"

TonePlayer piezo(PIEZO);
```

```
Button     nextBtn(BTN_NEXT);

// Define note sequences
const ToneNote ALERT[] = {
  {880, 100}, {0, 50}, {880, 100}, {0, 50}, {880, 200}
};

const ToneNote STARTUP[] = {
  {262, 150}, {330, 150}, {392, 150}, {523, 300}
};

const ToneNote CONFIRM[] = {
  {523, 80}, {659, 80}, {784, 200}
};

void setup() {
  Serial.begin(115200);
  piezo.begin();
  nextBtn.begin();
  pinMode(LED_GREEN, OUTPUT);
  digitalWrite(LED_GREEN, HIGH);
  piezo.play(STARTUP, sizeof(STARTUP) / sizeof(STARTUP[0]));
  Serial.println("Press NEXT to play ALERT. Long press for CONFIRM.");
}

void loop() {
  piezo.update();  // Must call every loop pass

  Button::Event ev = nextBtn.read();
  if (ev == Button::SHORT_PRESS && !piezo.isPlaying()) {
    piezo.play(ALERT, sizeof(ALERT) / sizeof(ALERT[0]));
    Serial.println("ALERT played.");
  }
  if (ev == Button::LONG_PRESS && !piezo.isPlaying()) {
    piezo.play(CONFIRM, sizeof(CONFIRM) / sizeof(CONFIRM[0]));
    Serial.println("CONFIRM played.");
  }
}
```

Keep TonePlayer.h

Save this file alongside Button.h — you will use it in Tier 2 and Tier 3 projects for audio feedback on UI events. The pattern of update() called every loop() pass is identical to Button::read(). Both are non-blocking, both cost nothing when idle.

3 DAC audio — true analog output

The ESP32 has two built-in 8-bit DAC channels. DAC1 is on GPIO 25 — the DAC_AUDIO define on the E4. Unlike PWM which switches rapidly between HIGH and LOW, the DAC outputs a true analog voltage anywhere between 0V and 3.3V. This produces much smoother, more natural-sounding audio.

3.1 Basic DAC output

```
#include <Arduino.h>

void setup() {
  Serial.begin(115200);
  // No pinMode needed for DAC — it is always available on GPIO 25

  // Write a static voltage: 0=0V, 128=1.65V, 255=3.3V
  dacWrite(DAC_AUDIO, 128);   // ~1.65V midpoint
  Serial.println("DAC set to midpoint (1.65V)");
  delay(1000);

  dacWrite(DAC_AUDIO, 0);     // 0V
  Serial.println("DAC set to 0V");
}

void loop() {}
```

3.2 Generating a sine wave

A sine wave is the purest audio tone. To generate one, step through a pre-calculated sine table and write each value to the DAC at a controlled rate. The step rate determines the frequency.

```
#include <Arduino.h>
#include <math.h>

// Pre-calculate 256-point sine table at startup
uint8_t sineTable[256];

void buildSineTable() {
  for (int i = 0; i < 256; i++) {
    // sin() returns -1 to +1, scale to 0-255 for DAC
    sineTable[i] = (uint8_t)((sin(2.0 * PI * i / 256.0) + 1.0) * 127.5);
  }
}

// Generate a sine wave at targetHz for durationMs
// NOTE: This uses delay internally — for blocking tone only.
// Non-blocking version follows in Section 3.3.
void playSine(int targetHz, int durationMs) {
  // How many microseconds between each sample?
  // 256 samples per cycle, targetHz cycles per second
  unsigned long stepUs = 1000000UL / (targetHz * 256);
  unsigned long endMs  = millis() + durationMs;
  int idx = 0;
```

```
  while (millis() < endMs) {
    dacWrite(DAC_AUDIO, sineTable[idx]);
    idx = (idx + 1) & 0xFF;  // Wrap at 256
    delayMicroseconds(stepUs);
  }
  dacWrite(DAC_AUDIO, 128);  // Return to midpoint
}

void setup() {
  Serial.begin(115200);
  buildSineTable();

  Serial.println("Playing A4 (440Hz) sine wave for 1 second...");
  playSine(440, 1000);

  Serial.println("Playing C5 (523Hz) sine wave for 1 second...");
  playSine(523, 1000);

  Serial.println("Done.");
}

void loop() {}
```

Key point

The DAC produces a smoother, more pleasant tone than the piezo at the same frequency — but it is much quieter. Without amplification, the DAC output is suitable for headphones connected directly to the DAC_AUDIO header, or as input to an amplifier module. The piezo is better for audible alerts; the DAC is better for audio quality.

3.3 Other waveforms

The sine table approach works for any waveform. Here are three common ones:

```
uint8_t sawTable[256];    // Sawtooth — linearly rising then reset
uint8_t triTable[256];    // Triangle — linearly rising then falling
uint8_t sqrTable[256];    // Square — abrupt high/low

void buildWaveTables() {
  for (int i = 0; i < 256; i++) {
    sawTable[i] = (uint8_t)i;              // 0..255 linear rise
    triTable[i] = (i < 128)                // 0..255..0 triangle
                  ? (uint8_t)(i * 2)
                  : (uint8_t)((255 - i) * 2);
    sqrTable[i] = (i < 128) ? 255 : 0;    // Square: half high, half low
  }
}

// Use exactly like sineTable — swap the table name in playSine()
```

Each waveform has a distinct timbre — the quality of the sound beyond just its pitch. Sine = pure, smooth. Triangle = slightly hollow. Sawtooth = bright, buzzy. Square = harsh, electronic. Experiment with all four through the DAC header.

4 PWM transistor output

GPIO 12 drives the base of a BC847 NPN transistor via a current limiting resistor. The transistor's collector connects to the load — in the E4 course, the TFT backlight. The transistor acts as a switch that the ESP32 controls via PWM, allowing it to drive loads that draw more current than a GPIO pin can safely supply.

4.1 Why a transistor?

Parameter	Detail
ESP32 GPIO max current	12mA per pin, 40mA total across all GPIO. Exceeding this risks permanent damage.
TFT backlight current	Typically 40-80mA. Well beyond safe GPIO drive capability.
BC847 collector current	Up to 100mA — safely handles the backlight and many other loads.
GPIO role	Drives only the transistor base (~1mA). The transistor does the heavy current work.

4.2 PWM control of the transistor

The transistor is controlled identically to the LED dimming in T02 — LEDC PWM on channel 0. The only difference is the load being driven:

```
#include <Arduino.h>

const int BL_CHANNEL = 0;
const int BL_FREQ    = 5000;
const int BL_RES     = 8;

void setup() {
  Serial.begin(115200);

  // Always initialise LOW before LEDC on GPIO 12
  pinMode(TFT_BL, OUTPUT);
  digitalWrite(TFT_BL, LOW);

  ledcSetup(BL_CHANNEL, BL_FREQ, BL_RES);
  ledcAttachPin(TFT_BL, BL_CHANNEL);

  Serial.println("PWM ramp up 0->255 over 2 seconds");
  for (int b = 0; b <= 255; b++) {
    ledcWrite(BL_CHANNEL, b);
    delay(8);
  }

  Serial.println("Full brightness. Ramp down...");
  for (int b = 255; b >= 0; b--) {
    ledcWrite(BL_CHANNEL, b);
    delay(8);
  }

  ledcWrite(BL_CHANNEL, 0);
  Serial.println("Off.");
```

```
}

void loop() {}
```

GPIO 12 boot sensitivity reminder

Always set GPIO 12 LOW with digitalWrite() before calling ledcSetup(). This is especially important in projects that also use the TFT display. If GPIO 12 is HIGH at boot it can affect the flash voltage detection. The E4 schematic keeps this safe by design but your firmware must respect the initialisation order.

5 Tier 1 consolidation project

This final Tier 1 project brings together every skill from T01 through T07 into a single, complete, professionally structured firmware project. It is intentionally more complex than the individual exercises — it demonstrates what competent embedded firmware actually looks like.

The project is an environmental alert station. It reads the AHT20 temperature and LDR, displays readings on the OLED, plays audio alerts when thresholds are exceeded, and provides full button-driven control — all non-blocking, all millis()-timed, all using the Fundrum conventions.

5.1 Feature list

- AHT20: reads temperature and humidity every 5 seconds
- LDR: reads light level continuously with 8-sample averaging
- OLED: two-page display — live readings with bar graph, and alert thresholds
- Piezo: startup melody, alert tone when temperature or humidity exceeds threshold
- LEDs: GREEN = running, BLUE = alert active, RED = sensor error
- NEXT short: cycle pages. NEXT long: force immediate sensor read
- ENT short: acknowledge and silence active alert. ENT long: reset min/max statistics
- Fully non-blocking — every event uses millis() or state machines

5.2 Project structure

Create a new project called E4_T07_AlertStation with the following files in src/:

File	Purpose
`main.cpp`	Main firmware — setup(), loop(), state management
`Button.h`	Copy from T03 — debounced button class
`TonePlayer.h`	Copy from T07 Section 2.3 — non-blocking tone sequencer

5.3 main.cpp

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_AHTX0.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "Button.h"
#include "TonePlayer.h"

// ── Display ──────────────────────────────────────────────
#define OLED_WIDTH  128
#define OLED_HEIGHT  64
#define OLED_RESET  -1
#define OLED_ADDR  0x3C

Adafruit_SSD1306 oled(OLED_WIDTH, OLED_HEIGHT, &Wire, OLED_RESET);
Adafruit_AHTX0   aht;
Button           nextBtn(BTN_NEXT);
Button           entBtn(BTN_ENT);
```

```
TonePlayer       piezo(PIEZO);

// ── Alert thresholds ──────────────────────────────────────
const float TEMP_HIGH_C   = 28.0;
const float HUMID_HIGH_PCT = 70.0;

// ── State ─────────────────────────────────────────────────
enum Page { PAGE_LIVE, PAGE_THRESHOLDS, PAGE_COUNT };
Page currentPage = PAGE_LIVE;

float tempC    = 0.0, humidity = 0.0;
float minTemp  = 999.0, maxTemp = -999.0;
float minHumid = 999.0, maxHumid = -999.0;
int   lightPct = 0;
bool  alertActive = false;

unsigned long lastRead   = 0;
const unsigned long READ_INTERVAL = 5000;

// ── Note sequences ────────────────────────────────────────
const ToneNote STARTUP[]  = {{262,120},{330,120},{392,120},{523,250}};
const ToneNote ALERT_SND[]=
{{880,80},{0,40},{880,80},{0,40},{880,80},{0,40},{880,200}};
const ToneNote ACK_SND[]  = {{523,80},{392,80},{262,200}};

// ── LDR ───────────────────────────────────────────────────
int readLDR() {
  long t = 0;
  for (int i = 0; i < 8; i++) { t += analogRead(LDR); delay(2); }
  return constrain(map(t/8, 200, 3800, 0, 100), 0, 100);
}

// ── Alert handling ────────────────────────────────────────
void checkAlerts() {
  bool triggered = (tempC > TEMP_HIGH_C) || (humidity > HUMID_HIGH_PCT);
  if (triggered && !alertActive) {
    alertActive = true;
    digitalWrite(LED_BLUE, HIGH);
    piezo.play(ALERT_SND, sizeof(ALERT_SND)/sizeof(ALERT_SND[0]));
    Serial.printf("ALERT! Temp:%.1fC Humid:%.1f%%\n", tempC, humidity);
  }
  if (!triggered && alertActive) {
    alertActive = false;
    digitalWrite(LED_BLUE, LOW);
  }
}

// ── OLED pages ────────────────────────────────────────────
void drawBarH(int x, int y, int w, int h, int pct) {
  oled.drawRect(x, y, w, h, SSD1306_WHITE);
  int fw = map(constrain(pct,0,100), 0, 100, 0, w-2);
  if (fw > 0) oled.fillRect(x+1, y+1, fw, h-2, SSD1306_WHITE);
}
```

```
void drawPageLive() {
  oled.clearDisplay();
  // Header
  oled.fillRect(0,0,128,13,SSD1306_WHITE);
  oled.setTextColor(SSD1306_BLACK);
  oled.setTextSize(1);
  oled.setCursor(2,3);
  oled.print(alertActive ? "! ALERT ACTIVE !" : "E4 Alert Station");
  // Temp
  oled.setTextColor(SSD1306_WHITE);
  oled.setTextSize(1);
  oled.setCursor(0,16);
  oled.print("Tmp");
  oled.setTextSize(2);
  oled.setCursor(24,14);
  oled.printf("%.1fC", tempC);
  // Humidity
  oled.setTextSize(1);
  oled.setCursor(0,33);
  oled.print("Hum");
  oled.setTextSize(2);
  oled.setCursor(24,31);
  oled.printf("%.1f%%", humidity);
  // Light bar
  oled.setTextSize(1);
  oled.setCursor(0,52);
  oled.printf("Lgt%3d%%", lightPct);
  drawBarH(42, 52, 86, 10, lightPct);
  oled.display();
}

void drawPageThresholds() {
  oled.clearDisplay();
  oled.fillRect(0,0,128,13,SSD1306_WHITE);
  oled.setTextColor(SSD1306_BLACK);
  oled.setTextSize(1);
  oled.setCursor(2,3);
  oled.print("Thresholds  2/2");
  oled.setTextColor(SSD1306_WHITE);
  oled.setCursor(0,16);
  oled.printf("Temp  alert >%.1fC", TEMP_HIGH_C);
  oled.setCursor(0,26);
  oled.printf("Hum   alert >%.0f%%", HUMID_HIGH_PCT);
  oled.setCursor(0,38);
  oled.printf("Min T:%.1f Max T:%.1f", minTemp, maxTemp);
  oled.setCursor(0,48);
  oled.printf("Min H:%.1f Max H:%.1f", minHumid, maxHumid);
  oled.display();
}

void refreshPage() {
  switch(currentPage) {
    case PAGE_LIVE:       drawPageLive();       break;
    case PAGE_THRESHOLDS: drawPageThresholds(); break;
```

```
    default: break;
  }
}

// ── Sensor read ──────────────────────────────────────────
void readSensors() {
  sensors_event_t h, t;
  aht.getEvent(&h, &t);
  tempC    = t.temperature;
  humidity = h.relative_humidity;
  lightPct = readLDR();
  if (tempC    < minTemp)   minTemp   = tempC;
  if (tempC    > maxTemp)   maxTemp   = tempC;
  if (humidity < minHumid) minHumid  = humidity;
  if (humidity > maxHumid) maxHumid  = humidity;
  checkAlerts();
  refreshPage();
  Serial.printf("T:%.1fC H:%.1f%% L:%d%%  Alert:%s\n",
                tempC, humidity, lightPct, alertActive?"YES":"no");
}

// ── Setup ────────────────────────────────────────────────
void setup() {
  Serial.begin(115200);
  Serial.printf("T07 Alert Station  FW:%s  Date:%s\n",
                FW_VERSION, FW_DATE);

  Wire.begin(I2C_SDA, I2C_SCL);

  pinMode(LED_GREEN, OUTPUT); digitalWrite(LED_GREEN, LOW);
  pinMode(LED_RED,   OUTPUT); digitalWrite(LED_RED,   LOW);
  pinMode(LED_BLUE,  OUTPUT); digitalWrite(LED_BLUE,  LOW);

  nextBtn.begin();
  entBtn.begin();
  piezo.begin();

  bool ok = true;
  if (!oled.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
    Serial.println("ERROR: OLED"); ok = false;
  }
  if (!aht.begin(&Wire)) {
    Serial.println("ERROR: AHT20"); ok = false;
  }
  if (!ok) { digitalWrite(LED_RED, HIGH); while(true){delay(100);} }

  digitalWrite(LED_GREEN, HIGH);
  piezo.play(STARTUP, sizeof(STARTUP)/sizeof(STARTUP[0]));
  readSensors();
  Serial.println("Ready. NEXT=page/read  ENT=ack/reset");
}

// ── Loop ─────────────────────────────────────────────────
void loop() {
```

```
  unsigned long now = millis();
  piezo.update();

  Button::Event nextEv = nextBtn.read();
  Button::Event entEv  = entBtn.read();

  // NEXT short: cycle page
  if (nextEv == Button::SHORT_PRESS) {
    currentPage = (Page)((currentPage + 1) % PAGE_COUNT);
    refreshPage();
  }

  // NEXT long: force immediate read
  if (nextEv == Button::LONG_PRESS) {
    lastRead = now;
    readSensors();
    Serial.println("Forced read.");
  }

  // ENT short: acknowledge alert
  if (entEv == Button::SHORT_PRESS && alertActive) {
    piezo.play(ACK_SND, sizeof(ACK_SND)/sizeof(ACK_SND[0]));
    piezo.stop();  // Cancel alert tone if still playing
    digitalWrite(LED_BLUE, LOW);
    alertActive = false;
    Serial.println("Alert acknowledged.");
    refreshPage();
  }

  // ENT long: reset min/max statistics
  if (entEv == Button::LONG_PRESS) {
    minTemp = maxTemp = tempC;
    minHumid = maxHumid = humidity;
    Serial.println("Statistics reset.");
    refreshPage();
  }

  // Timed sensor read
  if (now - lastRead >= READ_INTERVAL) {
    lastRead = now;
    readSensors();
  }
}
```

5.4 What to observe

- Startup melody plays on power-up — confirms piezo, OLED, AHT20, and LEDs all working
- Page 1 shows live temperature, humidity, and light bar graph — all updating every 5 seconds
- Page 2 shows alert thresholds and min/max statistics accumulated since last reset
- Breathe on the AHT20 — if humidity exceeds 70% the alert fires, BLUE LED lights, and the alert tone plays
- ENT short press acknowledges the alert silently with a descending confirm tone
- ENT long press resets the min/max statistics — visible immediately on page 2

- NEXT long press forces an immediate read outside the 5-second schedule
- Nothing blocks — button presses respond instantly even during tone playback

You have completed Tier 1

This project uses millis() timing, build_flags GPIO convention, Button.h debounce and short/long press, TonePlayer.h non-blocking audio, AHT20 I2C sensor, LDR analog ADC1, OLED zone layout with multi-page navigation, and LED status indicators. Every single skill from T01 through T07 is present and working together. That is a complete embedded firmware skill set.

6 Tier 1 complete — what you have learned

You have covered the full Tier 1 curriculum. Here is a consolidated reference of every skill, pattern, and convention you now own:

T	Topic	Skills mastered
T01	PlatformIO and the E4	Toolchain stack, platformio.ini, build_flags, millis() non-blocking timing, Serial debugging
T02	GPIO and build_flags	Output/input/pullup modes, input-only pins, boot-sensitive pins, ADC1 rule, LEDC PWM dimming
T03	Buttons and debouncing	Contact bounce, millis() debounce, edge detection, short/long press, Button.h class, state machines
T04	I2C fundamentals	SDA/SCL bus, 7-bit addressing, pull-ups, I2C scanner, AHT20 reading, multiple devices, Wire.begin()
T05	OLED display	SSD1306 128×64, display buffer, zone layout, drawing primitives, partial erase, multi-page navigation
T06	One-wire and analog	DS18B20 one-wire, ROM addressing, non-blocking conversion state machine, LDR voltage divider, ADC averaging, bar graph
T07	Audio output basics	LEDC piezo tones, TonePlayer.h non-blocking sequencer, DAC sine/triangle/sawtooth waves, PWM transistor load control

What's next — Tier 2

Tier 2 builds on every Tier 1 skill and introduces the tools used in real production firmware:

- T08 — TFT display and sprite pipeline: the 2.8" ILI9341 with TFT_eSPI, User_Setup.h, sprite double-buffering, smooth fonts, and multi-zone colour layouts
- T09 — Touch input: XPT2046 calibration, touch zones, SPI bus sharing with the TFT
- T10 — SD card and file I/O: SPI init order, CSV logging, BMP splash screens from SD
- T11 — NVS persistence: Preferences library, reboot-safe settings, key length rules
- T12 — WiFi and MQTT: connect, publish, subscribe, Node-RED dashboard integration
- T13 — LittleFS: partition config, file upload, web dashboard, config files
- T14 — OTA updates: ElegantOTA workflow, version display, update-safe firmware structure
- T15 — ESP-NOW: peer-to-peer without infrastructure, Fundrum protocol, mesh basics
- T16 — I2S audio and MEMS mic: ICS43434 microphone, FFT analysis, real-time spectrum